

eSal — NFC Tap-to-Pair / Tap-to-Claim

Everything the iOS app needs to implement so that tapping the store's NFC tag opens eSal directly and completes the right action.

The one idea to hold onto through all of this: there is exactly **one physical NFC tag** per store, permanently stuck next to the receipt printer's antenna. That tag's content is a plain HTTPS link that changes over its lifetime — first it says "pair me," later it says "claim a receipt." Your app never talks to the tag directly. iOS reads it for you and, once Universal Links are set up, hands your app the URL. Your job is: receive that URL, read two query parameters off it, and call one of two backend endpoints.

Table of Contents

1. The big picture
2. The exact URLs the tag will ever contain
3. Step 1 — Universal Links setup in Xcode
4. Step 2 — Receiving and parsing the URL
5. Step 3 — The two backend calls, in full
6. Edge cases and UI states to handle
7. Testing checklist

1. The big picture

Here is the full sequence of events, end to end, so the two backend calls below make sense in context:

- 1 A shop owner sets up a new receipt device.** The Raspberry Pi inside it writes a link onto the NFC tag containing a random `pairingId` and `mode=pair`.
- 2 The shop owner opens the eSal app and taps their phone to the tag.** iOS reads the tag, recognizes the domain as belonging to eSal (because of the Universal Links setup in Step 1 below), and launches your app with that URL

instead of opening a browser. Your app reads `mode=pair`, and calls the backend's *pair* endpoint using the shop owner's own login. This links that physical device to their store.

3 The Raspberry Pi notices pairing succeeded within a few seconds (it's been polling in the background) and — this is important — **rewrites the tag itself**, replacing `mode=pair` with `mode=claim`, same `pairingId`. This happens automatically, once, and you don't need to do anything for it.

4 From then on, any customer who buys something taps the same tag to claim their receipt into their own account. Your app reads `mode=claim` this time, and calls the backend's *claim* endpoint using the customer's own login.

Notice the tag only ever has **one** mode live on it at any given time — never both. But your app can't know in advance which one it'll read, since it depends entirely on whether that store's device has completed pairing yet. **Your code must handle both cases every time a tap comes in**, branching purely on the `mode` value you read out of the URL.

2. The exact URLs the tag will ever contain

Both forms share the same host and path — only `mode` changes:

Situation	URL on the tag
Before a device is paired	<code>https://esal.onrender.com/nfc?pairingId=<uuid>&mode=pair</code>
After pairing succeeds (auto-rewritten by the Pi)	<code>https://esal.onrender.com/nfc?pairingId=<uuid>&mode=claim</code>

`<uuid>` is a standard UUID string, e.g. `689a92db-5bc5-4693-b4b6-dda4665398ce`. It's a completely opaque identifier as far as your app is concerned — you just read it out and pass it straight through to the backend call, you never need to parse or validate its format yourself.

There is no real webpage at `/nfc` . That path doesn't need to (and currently doesn't) return anything meaningful over plain HTTP — it exists purely as a URL pattern for Universal Links to match against. Once your app is correctly wired up, iOS intercepts the tap before any network request to `/nfc` is ever made, so this is not something you need to worry about or treat as a bug.

3. Step 1 — Universal Links setup in Xcode

This is the part that makes iOS say "**Open in eSal**" instead of handing the tag off to Safari/whatever the default browser is. Without this step, tapping the tag will just open the URL in a browser and nothing in your app runs at all.

3.1 — Identifiers you'll need (already registered on the Apple Developer account)

Identifier	Value
Team ID	Z44K6B6BA2
Bundle ID	com.Raghad.eSal
Domain	esal.onrender.com

Double check your Xcode project's target Bundle Identifier is set to exactly `com.Raghad.eSal` — Universal Links matching fails silently (no error, it just quietly falls back to Safari) if this doesn't match what the backend serves.

3.2 — Add the Associated Domains capability

In Xcode: select your app target → **Signing & Capabilities** tab → + **Capability** → **Associated Domains**. Add exactly one entry:

```
applinks:esal.onrender.com
```

This requires a real provisioning profile tied to the `com.Raghad.eSal` bundle ID — the free/personal-team signing option does support Associated Domains, but you do need to build and run on a **real device**, not the Simulator, for the actual NFC tap to work (the Simulator has no NFC hardware at all).

3.3 — What the backend already serves (or will serve) for this to work

The backend hosts a file at exactly this address, with no file extension:

```
https://esal.onrender.com/.well-known/apple-app-site-association
```

Its content, so you know what to expect if you ever fetch it yourself to debug:

```
{
  "applinks": {
    "apps": [],
    "details": [
      {
        "appID": "Z44K6B6BA2.com.Raghad.eSal",
        "paths": ["/nfc*"]
      }
    ]
  }
}
```

You don't need to do anything with this file yourself — it's entirely a backend/Apple concern. It's listed here purely so you can sanity-check that the `appID` matches your project's Team ID + Bundle ID if something isn't matching.

Caching gotcha you will very likely hit while testing: iOS fetches and caches this file once, at the point your app (with the Associated Domains entitlement) gets installed or re-installed on a device. If the backend team deploys this file *after* you've already installed a test build, your device won't know about it yet. **Delete the app and reinstall it** (or do a clean install from Xcode) any time the backend's AASA content changes, or any time Universal Links aren't matching for no obvious reason.

3.4 — Quick way to test Universal Link matching without an NFC tag at all

Open the **Notes** app on the test iPhone, type out `https://esal.onrender.com/nfc?pairingId=test&mode=pair` as plain text, then **long-press** the link that Notes auto-detects. If Associated Domains is wired up correctly, you'll see **"Open in eSal"** in the preview. If you instead see a Safari preview, something in this section isn't matching yet — check the bundle ID, the entitlement, and whether you've reinstalled since the AASA file last changed.

4. Step 2 — Receiving and parsing the URL

Once Universal Links are matching, iOS launches (or resumes) your app and hands you the URL. In SwiftUI, the simplest way to receive it is `.onOpenURL` on your top-level view:

```
WindowGroup {
    ContentView()
        .onOpenURL { url in
            handleIncomingTap(url)
        }
}
```

Parse the two values you need with `URLComponents` — do not try to parse the query string by hand:

```

func handleIncomingTap(_ url: URL) {
    guard
        let components = URLComponents(url: url, resolvingAgainstBaseURL: false),
        let queryItems = components.queryItems,
        let pairingId = queryItems.first(where: { $0.name == "pairingId" })?.value,
        let mode = queryItems.first(where: { $0.name == "mode" })?.value
    else {
        // Malformed link – see "Edge cases" section below
        return
    }

    switch mode {
    case "pair":
        handlePairingTap(pairingId: pairingId)
    case "claim":
        handleClaimTap(pairingId: pairingId)
    default:
        // Unknown mode value – see "Edge cases" section below
        break
    }
}

```

That's the entire parsing job — two string values off the query string, nothing else on the URL matters to you.

5. Step 3 — The two backend calls, in full

Both endpoints require the current user to already be logged in — attach the app's normal JWT the exact same way you do for every other eSal API call (`Authorization: Bearer <token>`).

5.1 — `mode=pair` → shop owner is pairing a device

**Method /
Path**

POST `/devices/pair/nfc`

Auth required	Yes — logged-in shop owner account (<code>accountType: "SHOP"</code>) with a store already assigned
Request body	<pre>{ "pairingId": "<the pairingId from the URL>" }</pre>
Success response	The created device record, e.g.: <pre>{ "id": "clx...", "pairingId": "689a92db-5bc5-4693-b4b6-dda4665398ce", "storeId": "clx...", "apiKey": "...", "apiKeyRetrieved": false, "createdAt": "2026-07-15T10:00:00.000Z" }</pre> Your app doesn't need to do anything with these fields beyond confirming the call succeeded — the <code>apiKey</code> here is for the Pi, not for your app.
403 Forbidden	Logged-in user isn't a shop owner, or has no store yet. Message: <i>"Only shop accounts with a store can pair devices."</i> Show something like "This action is only available to store accounts."
409 Conflict	This exact <code>pairingId</code> has already been paired before (someone tapped the tag twice, or it's a stale/reused tag). Message: <i>"This device has already been paired."</i> Show "This device is already set up."

5.2 — `mode=claim` → **customer is claiming a receipt**

Method / Path	<code>POST /receipts/claim/nfc</code>
Auth required	Yes — logged-in customer account (<code>accountType: "CUSTOMER"</code>)
Request body	<pre>{ "pairingId": "<the pairingId from the URL>" }</pre>

Success response	The now-claimed receipt, e.g.: <pre> { "id": "clx...", "deviceId": "clx...", "userId": "clx...", "status": "ASSIGNED", "total": "42.50", "createdAt": "2026-07-15T10:05:00.000Z", ... }</pre> Note <code>total</code> and other money fields are serialized as strings, not numbers — same as everywhere else in the API, convert with <code>Decimal / Double(...)</code> as you already do elsewhere. This is the receipt that should now show up in the customer's receipt list — good moment to refresh that screen or navigate straight to the receipt detail view.
403 Forbidden	Logged-in user isn't a customer account. Message: <i>"Only Customer Accounts can claim receipts."</i> Show "This action is only available to customer accounts."
404 Not Found (device)	The <code>pairingId</code> doesn't match any known device at all. Message: <i>"Device Not Found."</i> Should be rare/only happen with a malformed or fake link.
404 Not Found (no receipt)	The device exists, but has no receipt currently waiting to be claimed. Message: <i>"No Pending receipt found for this device."</i> This is the most common real-world case to design for — e.g. the customer tapped before the cashier finished ringing them up, or they tap again after already claiming. Show something friendly like "No new receipt to claim yet — ask the cashier to complete your purchase first."

6. Edge cases and UI states to handle

User isn't logged in when the tap comes in. Both endpoints require a JWT — if there's no logged-in session, don't attempt the call at all. Recommended: store

the incoming URL, route to your login screen, and re-run `handleIncomingTap` once login completes.

Wrong account type taps the wrong mode. A customer account tapping a `mode=pair` tag (device not yet paired), or a shop owner tapping a `mode=claim` tag, both come back as **403** from the backend — see the tables above for exact messages. Don't treat these as generic network errors; show the specific friendly message.

Malformed URL / missing query params. Should not normally happen from the real tag, but defend against it anyway — if `pairingId` or `mode` is missing, or `mode` is some unrecognized value, just no-op silently or show a generic "couldn't read that tag" message rather than crashing.

Tap arrives while the app is backgrounded vs. fully closed. Both cases route through the same `.onOpenURL /scene-delegate` mechanism once iOS finishes launching or resuming your app — no special-casing needed on your end.

7. Testing checklist

1. Bundle ID in Xcode is exactly `com.Raghad.eSal` .
2. Associated Domains capability added, with `applinks:esal.onrender.com` .
3. Built and running on a **real device** (Simulator has no NFC reader).
4. Notes-app long-press test (section 3.4) shows "Open in eSal", not a Safari preview.
5. If it doesn't: reinstall the app fresh, then retry — this fixes the large majority of "Universal Links just aren't matching" cases during active development.
6. Physically tap the real tag with the app installed and logged in as a shop owner → confirm pairing succeeds and the Pi's LED/terminal shows a successful pair.
7. Have the Pi (or the backend team) trigger a pending receipt on that now-paired device, then tap again logged in as a customer → confirm the receipt appears claimed in the app.